MODULE 4

Claude Code Deep Dive: CLI & VS Code

Duration	Format	Audience	Day / Position
45 min	**Lecture + Demo**	**Mixed Engineers**	**Day 1 / Fourth**

LEARNING OBJECTIVES

By the end of this module, participants will be able to:

1. Navigate Claude Code's terminal-first workflow: start sessions, use interactive mode, and apply keyboard shortcuts for efficient development
2. Use essential slash commands (/init, /compact, /clear, /model, /config, /plan) to manage sessions, context, and model selection
3. Configure and use the VS Code extension for real-time inline diffs, sidebar panel navigation, and checkpoint management
4. Apply the checkpoint and /rewind system to safely experiment with code changes and recover from unwanted modifications
5. Manage context effectively using @-references, /context inspection, and token budget awareness to keep Claude focused
6. Configure the permission model (allow-lists, deny rules, session-only trust) and understand settings.json hierarchy for team-safe deployments

SESSION TIMING BREAKDOWN

0:00 – 0:10	**Section 1: Terminal-First Workflow — Interactive Mode & Essentials**
0:10 – 0:20	**Section 2: Slash Commands — Your Session Control Panel**
0:20 – 0:30	**Section 3: VS Code Extension — IDE-Integrated AI**
0:30 – 0:37	**Section 4: Checkpoints & /rewind — Your Safety Net**
0:37 – 0:42	**Section 5: Permission Model & Settings Hierarchy**
0:42 – 0:45	**Wrap-Up: Quick Reference & Transition to Module 5**

0:00 – 0:10	SECTION 1: Terminal-First Workflow — Interactive Mode & Essentials

1. Terminal-First Workflow: Interactive Mode & Essentials

1.1 Why Terminal-First?

Claude Code was designed as a terminal-native tool. Unlike browser-based AI assistants or IDE plugins that overlay on top of your editor, Claude Code lives where you already work — in the command line. This is a deliberate architectural choice:

Terminal Advantage	Why It Matters for TI Engineers
Direct filesystem access	Claude reads your project files, runs your tests, executes your scripts — no copy-paste
Git integration	Creates branches, commits, generates PR descriptions without leaving the session
Shell command execution	Runs pytest, linting, builds, and any CLI tool as part of the coding workflow
No context switching	Stay in your terminal workflow; no browser tabs or IDE panels to manage
Scriptable (claude -p)	Integrate into CI/CD pipelines, cron jobs, and automation scripts
Full codebase awareness	Automatically reads CLAUDE.md, understands project structure and dependencies

1.2 Starting a Session

There are several ways to launch Claude Code, each suited to different workflows:

```
# Interactive mode (most common) — opens a chat session
claude

# Start with an initial prompt
claude "Explain the structure of this project"

# Continue the last conversation (preserves context)
claude -c

# Resume a specific session by ID
claude --resume <session-id>

# One-shot query (non-interactive, returns result and exits)
claude -p "What does the main() function in app.py do?"

# Start with additional directory context
claude --add-dir ../shared-libs
```

☆ **KEY POINT:** Always start Claude Code from your project root directory (where CLAUDE.md lives). Claude automatically reads the directory structure, CLAUDE.md files, and recent git history to build context. Starting from the wrong directory means Claude lacks the project context it needs.

1.3 The Interactive Interface

Once inside an interactive session, you're in a conversational loop with Claude. Here's what you see and can do:

Interface Element	Description
Prompt (>)	Type your natural language instructions here. Multi-line input supported.
Status bar	Shows current model, token usage, and session info at the top
Diff output	When Claude edits files, you see a color-coded diff (green = added, red = removed)
Permission prompts	Claude asks before executing commands or writing files (configurable)
Tool usage indicators	Shows which tools Claude is using: Read, Write, Bash, Grep, etc.

1.4 Essential Keyboard Shortcuts

These shortcuts save significant time during interactive sessions. Memorize the top five:

Shortcut	Action	When to Use
Ctrl+C	Cancel current generation	Claude is going in the wrong direction
Ctrl+R	Search prompt history	Reuse or modify a previous prompt
Esc, Esc (double)	Open rewind/checkpoint menu	Undo Claude's last changes
Shift+Tab	Cycle permission modes	Switch between ask/auto-accept/plan mode
Cmd+T / Ctrl+T	Toggle extended thinking	Enable deeper reasoning for complex tasks
Tab	Accept autocomplete suggestion	Complete file paths and command names
! (exclamation)	Run shell command directly	Quick shell access without leaving session

1.5 The ! Shell Escape

You can run any shell command without leaving Claude Code by prefixing it with an exclamation mark. This is faster than asking Claude to run it, because it bypasses the AI and executes directly:

```
# Run shell commands directly (no AI processing)
> !git status
> !python -m pytest tests/ -v
> !cat src/config.py
> !pip install pandas

# Compare with asking Claude (slower, uses tokens):
> Run git status and show me the output
```

☆ **KEY POINT:** Use ! for commands you know exactly. Use natural language prompts when you want Claude to interpret the result, take action based on it, or chain multiple steps together. The ! shortcut is a power move for experienced developers.

QUICK CHECK (1 minute)

- What command starts Claude Code in interactive mode? (claude)
- What shortcut cancels a generation in progress? (Ctrl+C)
- How do you run a shell command without AI processing? (prefix with !)

0:10 – 0:20	SECTION 2: Slash Commands — Your Session Control Panel

2. Slash Commands: Your Session Control Panel

Slash commands are built-in commands that control Claude Code's behavior during a session. They start with / and are not processed by the AI — they're direct instructions to the Claude Code application itself. Think of them as meta-commands that manage the session rather than generate code.

2.1 Essential Slash Commands

These are the commands you'll use in almost every session. Master these first:

Command	What It Does	When to Use
/help	Lists all available commands including custom ones	First time in a session; discovering new commands
/init	Scans project and generates a starter CLAUDE.md	New project setup; bootstrapping Claude awareness
/clear	Wipes conversation history completely	Switching to a different task; context is polluted
/compact	Summarizes conversation to free context window	Long sessions; Claude starts forgetting earlier work
/model	Switch between Claude models (Sonnet, Haiku, Opus)	Balance cost vs. quality; simpler task = smaller model
/config	Open configuration menu for thinking, model, etc.	Adjusting session settings; enabling extended thinking
/plan	Enter plan mode — Claude plans but doesn't execute	Complex tasks; review the approach before changes
/rewind	Open checkpoint menu to restore previous state	Undo unwanted changes; recover from bad generations
/context	Show what's currently in Claude's context window	Debugging; checking if important files are loaded
/permissions	View and modify tool permissions for this session	Tightening or loosening what Claude can do

☆ **KEY POINT:** The three most important slash commands for daily use are /compact (manage context), /clear (reset context), and /rewind (undo changes). These three commands solve 80% of "Claude is behaving strangely" issues.

2.2 /init — Bootstrap Your Project

The /init command is the first thing you run in any new project. It scans your codebase and generates a CLAUDE.md file that gives Claude the context it needs:

```
# In your project root, run:
> /init
```

```
# Claude will:
# 1. Scan your directory structure
# 2. Read package.json / pyproject.toml / requirements.txt
# 3. Detect your tech stack, testing framework, lint config
# 4. Generate a CLAUDE.md with project context
# 5. Ask if you want to review/edit before saving

# The generated CLAUDE.md typically includes:
# - Project description and tech stack
# - Build and test commands
# - Code style rules detected from config files
# - Directory structure overview
```

2.3 /compact vs. /clear — Context Management

Understanding the difference between these two commands is critical for productive long sessions:

/compact	/clear
Summarizes conversation, keeps key decisions	Completely resets — blank slate
Frees ~60–80% of context window	Frees 100% of context window
Claude remembers what you decided earlier	Claude has no memory of previous conversation
Use when context is getting full but task continues	Use when switching to a completely different task
Safe to use mid-task	Only use between tasks
Run when Claude starts forgetting instructions	Run when starting fresh work

```
# Scenario: You've been working on validation functions
# for 30 minutes. Claude starts losing context.

# GOOD: Compact to free space while keeping decisions
> /compact
# Claude: "Summarized conversation. Key context: working on
# sensor validation functions, using tuple return pattern,
# following CLAUDE.md standards."

# Then continue working:
> Now create the pressure validation function

# DIFFERENT SCENARIO: You're done with validation and
# want to work on the CSV export module instead.

# GOOD: Clear completely for a fresh start
> /clear
```

```
> Let's work on the CSV export module in @src/io/exporter.py
```

2.4 /model — Choosing the Right Model

Claude Code gives you access to multiple Claude models. Choosing the right one for each task saves tokens and improves quality:

Model	Best For	Trade-off
Sonnet 4.5 (default)	Most coding tasks, balanced quality/speed	Good default for 80% of work
Haiku 4.5	Simple tasks: rename, format, add imports	3x faster, 3x cheaper, less capable
Opus 4.5 / 4.6	Complex architecture, multi-file refactoring, debugging	Slower, more expensive, highest quality

```
# Switch models during a session:
> /model
# Interactive picker appears — select with arrow keys

# Or specify from CLI at startup:
claude --model opus
claude --model haiku

# Pro tip: Start with Sonnet, switch to Opus
# only for the hard parts, then switch back.
```

2.5 Custom Slash Commands

You can create your own slash commands by placing Markdown files in .claude/commands/ (project-level) or ~/.claude/commands/ (global). This is how teams standardize common workflows:

```
# Create a project-level custom command:
# File: .claude/commands/review.md

Review the code in $ARGUMENTS for:
- Type hint completeness
- Google-style docstring coverage
- Error handling patterns
- Edge cases and boundary conditions
- Adherence to project CLAUDE.md standards

Provide a numbered list of issues found,
sorted by severity (critical first).

# Usage in Claude Code:
> /review src/core/validators.py
```

TEAM USE CASE

- IT engineers: create /review, /test-gen, /pr-describe commands for your team
- Process engineers: create /data-check, /pipeline-validate, /sensor-schema commands

- Both: share commands via the .claude/commands/ directory in your git repo

0:20 – 0:30	SECTION 3: VS Code Extension — IDE-Integrated AI

3. VS Code Extension: IDE-Integrated AI

3.1 Why Use the VS Code Extension?

While Claude Code's CLI is powerful, the VS Code extension adds a visual layer that many developers prefer. It's not a replacement for the CLI — it's a complement. Some features are only available in the CLI, but the extension excels at visual diff review and checkpoint management.

Feature	CLI vs. VS Code Extension
Code diffs	CLI: text-based diff in terminal — VS Code: rich inline diffs with syntax highlighting
File navigation	CLI: use @-references and ! commands — VS Code: click to open files, see project tree
Checkpoints	CLI: /rewind command — VS Code: hover over messages, click rewind button
Permission modes	CLI: Shift+Tab to cycle — VS Code: click mode indicator at bottom of panel
Terminal access	CLI: native — VS Code: open integrated terminal, run claude there too
Model switching	CLI: /model command — VS Code: click / or type / for command menu
Diagnostics	CLI: paste errors manually — VS Code: shares IDE diagnostics automatically

3.2 Installing the Extension

The VS Code extension is available from the VS Code Extension Marketplace:

```
# Option 1: From VS Code
# Open Extensions panel (Ctrl+Shift+X / Cmd+Shift+X)
# Search: "Claude Code"
# Install the Anthropic-published extension

# Option 2: From command line
code --install-extension Anthropic.claude-code

# After installation:
# 1. A Claude icon appears in the VS Code sidebar
# 2. Click it to open the Claude panel
# 3. Sign in with your Anthropic account
# 4. Start typing prompts in the panel
```

3.3 The Sidebar Panel

The Claude Code sidebar in VS Code is your primary interaction point. It provides a chat-like interface with enhanced visual features:

Panel Feature	What It Does
Chat input	Type prompts at the bottom, just like the CLI. Supports @-references.
Message history	Scrollable conversation with expandable tool usage details
Inline diffs	When Claude edits a file, the diff appears in your editor with green/red highlighting
Rewind buttons	Hover over any message to see Fork / Rewind Code / Rewind Both buttons
Mode indicator	Bottom of input box shows current mode: Normal, Plan, or Auto-accept
Command menu (/)	Click / or type / in input to access slash commands and settings
Spark icon	Top-right of editor — quick access when you have a file open
@terminal:name	Reference terminal output by name (e.g., test results, error logs)

☆ **KEY POINT:** The VS Code extension and CLI share the same conversation history. You can start a conversation in VS Code and continue it in the CLI with claude --resume, or vice versa. This lets you switch between visual and terminal workflows seamlessly.

3.4 Permission Modes in VS Code

The mode indicator at the bottom of the Claude panel controls how much autonomy Claude has. Click it to switch between modes:

Mode	Behavior	Best For
Normal (default)	Claude asks permission before each file edit or command	Learning, code review, unfamiliar codebases
Plan Mode	Claude describes what it will do, waits for approval	Complex tasks, architectural changes, review-first
Auto-accept	Claude makes edits without asking — fastest workflow	Trusted tasks, rapid prototyping, personal projects

⚠ **WARNING:** Auto-accept mode is powerful but risky. Only use it when you're working on non-critical code, personal projects, or when you're confident Claude understands your intent. Always have git commits as a safety net before enabling auto-accept.

0:30 – 0:37	SECTION 4: Checkpoints & /rewind — Your Safety Net

4. Checkpoints & /rewind: Your Safety Net

4.1 How Checkpoints Work

Claude Code automatically creates a checkpoint before every change it makes to your code. This happens in the background — you don't need to do anything. Every time you send a prompt and Claude modifies files, a snapshot of the previous state is saved.

Checkpoint Feature	How It Works
Automatic tracking	Every prompt creates a checkpoint before Claude edits code — no manual action needed
Granular restore	Choose to rewind code only, conversation only, or both
Persistence	Checkpoints survive closing and reopening sessions (cleaned up after 30 days)
Independent of git	Works even in projects without git — accessible to all users
Scope	Only tracks changes made by Claude Code tools (Write, Edit). Manual edits and bash (!) commands are NOT tracked

⚠ **WARNING:** Checkpoints do NOT track changes made via bash commands (e.g., !rm file.py, !mv old.py new.py, !sed -i). Only changes made through Claude's built-in Write and Edit tools are captured. If you need to run destructive bash commands, commit to git first.

4.2 Using /rewind

When Claude makes changes you don't want, /rewind is your undo button:

```
# Open the rewind menu:
> /rewind

# Or press Esc twice in quick succession:
# Esc, Esc

# The menu shows a list of checkpoints with:
# - Timestamp of each prompt
# - Preview of what was changed
# - Three restore options per checkpoint:
#   1. Fork conversation from here (keep code, new branch)
#   2. Rewind code to here (restore files, keep conversation)
#   3. Rewind both (restore files AND conversation)
```

4.3 Rewind Strategies

Different situations call for different rewind approaches:

Scenario	Rewind Action	Why

Claude's approach was wrong	Rewind both	Start fresh from a clean state — new plan needed
Code is wrong but reasoning was good	Rewind code only	Keep the conversation context, try a different implementation
Want to explore an alternative	Fork from here	Keep current changes intact, try a parallel approach
Claude modified too many files	Rewind code to before the edit	Restore all files to the safe state
Several prompts went off track	Rewind both to earlier checkpoint	Go back multiple steps to a known-good state

4.4 Checkpoints + Git: Belt and Suspenders

Checkpoints complement git — they don't replace it. Think of checkpoints as your session-level undo (like Ctrl+Z for AI changes), while git is your project-level version control:

Checkpoints (Session-Level)	Git (Project-Level)
Automatic, every prompt	Manual commits
Rewind within a session	Revert across sessions
30-day retention	Permanent history
Tracks Claude's tool changes only	Tracks ALL changes (including manual)
Great for experimentation	Great for milestones and collaboration

☆ **KEY POINT:** Best practice: commit to git at meaningful milestones (e.g., after completing a function, passing tests, finishing a sub-task). Use checkpoints for rapid experimentation within those milestones. This gives you both fine-grained undo (checkpoints) and permanent history (git).

0:37 – 0:42	SECTION 5: Permission Model & Settings Hierarchy

5. Permission Model & Settings Hierarchy

5.1 Why Permissions Matter

Claude Code can read files, write files, execute shell commands, and modify your git repository. That's powerful, but it means you need to control what Claude can and cannot do — especially in a corporate environment like TI where code safety and data protection are paramount.

5.2 The Permission System

Claude Code's permission model has three tiers: allow (auto-approved), ask (requires confirmation), and deny (blocked completely):

Permission Tier	Behavior	Example
allow	Claude executes without asking	Read any file, Bash(git status), Bash(python -m pytest *)
ask (default)	Claude shows what it wants to do and waits for your approval	Write(*.py), Bash(pip install *)
deny	Claude cannot use this tool/pattern at all	Read(.env), Read(.env.*), Write(production.config.*)

```
# Configure permissions in settings.json:
{
  "permissions": {
    "allow": [
      "Read",
      "Bash(git *)",
      "Bash(python -m pytest *)",
      "Bash(ruff check *)"
    ],
    "deny": [
      "Read(.env)",
      "Read(.env.*)",
      "Write(./production.config.*)",
      "Bash(rm -rf *)"
    ]
  }
}
```

5.3 Settings File Hierarchy

Claude Code reads settings from multiple files. More specific files override more general ones:

Settings Level (lowest → highest priority)	Location & Purpose
1. Enterprise policy (admin-controlled)	Managed by IT admin; org-wide defaults and restrictions

2. User global	~/.claude/settings.json — your personal defaults for all projects
3. Project shared	.claude/settings.json — team-shared, committed to git
4. Project local	.claude/settings.local.json — your personal overrides, git-ignored
5. CLI flags	Command-line arguments override everything for that session

☆ **KEY POINT:** For TI teams: put shared standards in .claude/settings.json (committed to git) and personal preferences in .claude/settings.local.json (git-ignored). This ensures consistency across the team while allowing individual customization.

5.4 Practical Settings for TI Teams

Here's a recommended settings.json for a TI Python project that balances productivity with safety:

```
// .claude/settings.json (committed to git, shared by team)
{
  "model": "claude-sonnet-4-5-20250929",
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(git status)",
      "Bash(git diff *)",
      "Bash(git log *)",
      "Bash(python -m pytest *)",
      "Bash(ruff check *)",
      "Bash(mypy *)"
    ],
    "deny": [
      "Read(.env)",
      "Read(.env.*)",
      "Read(**/secrets/**)",
      "Write(./production.*)",
      "Bash(rm -rf *)",
      "Bash(curl *)",
      "Bash(wget *)"
    ]
  }
}
```

5.5 Hooks: Automated Quality Gates

Hooks are shell commands that run automatically at specific points in Claude's workflow. They're like git hooks but for Claude Code:

```
// In settings.json, add hooks:
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Write(*.py)",
        "hooks": [{
```

```
          "type": "command",
          "command": "ruff check $FILE --fix"
        }]
      }
    ]
  }
}

# This automatically runs ruff (Python linter) every time
# Claude writes a .py file. Lint errors are caught immediately,
# before you even review the code.
```

✅ PERMISSION CHECKLIST FOR TI PROJECTS

- ALLOW: Read, Grep, Glob for code exploration
- ALLOW: git status/diff/log for version control awareness
- ALLOW: pytest, ruff, mypy for automated quality checks
- DENY: .env files and secrets directories
- DENY: Production config files
- DENY: Destructive bash commands (rm -rf, curl, wget)
- HOOKS: Auto-lint on every Python file write
- HOOKS: Auto-format with black after edits (optional)

0:42 – 0:45	WRAP-UP: Quick Reference & Transition to Module 5

6. Quick Reference & Transition

MODULE 4 RECAP

- ✅ Terminal-first workflow: start sessions with claude, use interactive mode for conversation, ! for shell escapes, and keyboard shortcuts for speed.
- ✅ Slash commands control your session: /init to bootstrap, /compact to manage context, /clear to reset, /model to switch models, /rewind to undo.
- ✅ VS Code extension adds visual diffs, sidebar panel, and clickable checkpoint management — complements the CLI, doesn't replace it.
- ✅ Checkpoints are automatic safety nets: every prompt creates a snapshot. Use /rewind or Esc+Esc to restore. Checkpoints + git = belt and suspenders.
- ✅ Permission model protects your codebase: allow/ask/deny tiers, settings.json hierarchy, and hooks for automated quality gates.
- ✅ Custom slash commands standardize team workflows: just create .md files in .claude/commands/ and share via git.

QUICK REFERENCE CARD

Keep this reference handy during labs and exercises:

Feature	Command / Shortcut	What It Does
Start session	claude	Open interactive mode in project root
One-shot query	claude -p "query"	Non-interactive; returns result and exits
Continue session	claude -c	Resume the last conversation
Shell escape	! command	Run shell command directly (no AI)
Cancel	Ctrl+C	Stop current generation
Rewind	Esc+Esc or /rewind	Open checkpoint restore menu
Compact	/compact	Summarize conversation, free context
Clear	/clear	Full context reset
Model switch	/model	Pick Sonnet, Haiku, or Opus
Plan mode	/plan	Claude plans but doesn't execute
Permission cycle	Shift+Tab	Normal → Plan → Auto-accept
Extended thinking	Cmd+T / Ctrl+T	Toggle deep reasoning mode
Prompt history	Ctrl+R	Search and reuse previous prompts
Context inspection	/context	See what's in Claude's memory
Custom commands	.claude/commands/*.md	Team-shared slash commands

TRANSITION TO MODULE 5

You now know how to drive Claude Code — the interface, the commands, the safety features. In Module 5, we'll shift to automation and scripting: using Claude Code's headless mode (claude -p) to integrate AI into your existing workflows, pipelines, and batch processing tasks. This is where process engineers will see the biggest productivity gains — automating repetitive Python scripting tasks that currently eat hours of your week.